MST and shortest-path solutions

A08-M01 | Cuts and ties

δ(S)={BD2,CD2,CE3,BF5}. AB and AC stay inside S; DE stays outside. The cut respects A. BD and CD are both light and either is safe for A. BD need not belong to every MST, since CD can replace it. Individual safety does not authorize adding all safe edges together.

A08-M02 | Complete an exchange

T+AC creates A–B–C–A. Choose f=BC, which crosses S={C} and has weight 1, equal to AC. The respecting cut guarantees f∉A. T′=T−BC+AC is a tree containing A∪{AC}; its weight is 2−1+1=2. Without respect, f could belong to A and the exchange could destroy A⊆T′. Another optimum alone would not prove safety for A.

A08-M03 | Strict cycle property

Any two edges of a unit triangle form a weight-two MST, so every tied maximum edge occurs in some MST. For uniquely heaviest e, assume T contains e. Removing e creates a cut crossed by another edge f of the cycle, with w(f)<w(e). T−e+f is cheaper, a contradiction. With ties, only the existence of an MST avoiding a designated maximum edge follows.

A08-M04 | Every Kruskal round

Accepted edges are AB−2,DE−1,AC1,BD2,EF2. Scan order is AB,DE,AC,BC,BD,CD,EF,CE,DF,BF. Components after each round are AB/C/D/E/F; AB/C/DE/F; ABC/DE/F; ABC/DE/F; ABCDE/F; ABCDE/F; ABCDEF; ABCDEF; ABCDEF; ABCDEF. Rejection paths are B–A–C for BC, C–A–B–D for CD, C–A–B–D–E for CE, D–E–F for DF, B–D–E–F for BF. Round 7 permits early termination.

A08-M05 | Every Prim round

Initially A is 0/∅ and all others ∞/∅. After A, Q is B−2/A,C1/A,D∞/∅,E∞/∅,F∞/∅; after B, C1/A,D2/B,F5/B,E∞/∅; after C, D2/B,E3/C,F5/B; after D, E−1/D,F4/D; after E, F2/E; after F, empty. Before D, S=ABC and crossing edges are BD2,CD2,CE3,BF5. BD and CD are light. Strict improvement keeps parent[D]=B on the equal CD candidate. A key is an edge weight with no required monotonicity, so DE=−1 decreases E.

A08-M06 | Negative and disconnected

Components are ABC and DEF, so n=6,c=2 and an MSF has four edges. {AB,AC,DE,EF} weighs −2+1−1+2=0; BC may replace AC. Prim makes an extracted ∞ vertex a new root with no parent edge, then scans its neighbors. Every spanning forest connects each original component and has n−c edges. Adding 10 to every weight adds 40 to every candidate total, preserving all optima.

A08-M07 | Optimality and uniqueness

The four sets are {AB,DE,EF,AC,BD}, {AB,DE,EF,AC,CD}, {AB,DE,EF,BC,BD}, {AB,DE,EF,BC,CD}, each of weight 2. AB is uniquely light for cut {A}, DE for {E}, and EF for {F}, hence all three are mandatory. Choose one edge from each of AC/BC and BD/CD. check.py enumerates 252 five-edge subsets to verify completeness. A triangle XY1,YZ1,XZ2 has repeated weights but the unique MST {XY,YZ}.

A08-M08 | Implementation costs

Indexed binary heaps with adjacency lists take n extracts and at most m decreases. With a position map and O(1) Q membership, time is O((n+m)log n). Lazy heaps push on improvements and discard stale records on pop; O(m) records and pops give O(n+m log(m+1)) time and O(n+m) auxiliary space. Array/matrix Prim scans n candidates and neighbors per round, $O(n^2)$. Fibonacci heaps use amortized O(1) decreases and O(log n) extracts, giving O(m+n log n). Kruskal uses O(m log m) comparison sorting plus O((n+m)(1+α(n))) DSU time. Lazy heaps retain obsolete records, so their size is not bounded by n.

Use graphs G+, G−, H, and N and the vertex/edge orders fixed in the slides. Slides contain every complete edge list.

A08-S01

δ(s,s)=0; δ(s,a)=δ(s,b)=δ(s,c)=−∞; δ(s,u)=δ(s,v)=∞. The cycle a→b→a costs −2, is reachable from s, and can reach c. Neither u nor v has any walk from s.

A08-S02

d[a]=3,π[a]=b and d[c]=6,π[c]=b; other entries are unchanged. Each new label concatenates an actual s-to-b walk with an actual arc, so it remains an actual walk cost and cannot fall below the infimum.

A08-S03

extract	d(s,a,b,c,t,z)	π(s,a,b,c,t,z)	Q after relaxation
init	(0, ∞, ∞, ∞, ∞, ∞)	(NIL, NIL, NIL, NIL, NIL, NIL)	s:0, a:∞, b:∞, c:∞, t:∞, z:∞
s	(0, 4, 1, ∞, ∞, ∞)	(NIL, s, s, NIL, NIL, NIL)	b:1, a:4, c:∞, t:∞, z:∞
b	(0, 3, 1, 6, ∞, ∞)	(NIL, b, s, b, NIL, NIL)	a:3, c:6, t:∞, z:∞
a	(0, 3, 1, 4, 10, ∞)	(NIL, b, s, a, a, NIL)	c:4, t:10, z:∞
c	(0, 3, 1, 4, 7, ∞)	(NIL, b, s, a, c, NIL)	t:7, z:∞
t	(0, 3, 1, 4, 7, ∞)	(NIL, b, s, a, c, NIL)	z:∞
z	(0, 3, 1, 4, 7, ∞)	(NIL, b, s, a, c, NIL)	

π[c] first becomes b, then a when a is extracted. π[t] first becomes a, then c when c is extracted. Vertex z stays ∞/NIL.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A08-S04

After s, Q=(a:2,b:5,t:∞); after a, Q=(t:4,b:5). Vertex t is settled at 4 before b reveals a=1. The true t distance is 3. The failed inequality is $\delta(s,b)=5 \leq \delta(s,a)=1$, which cannot hold with this negative suffix.

A08-S05

The successive d vectors are (0,4,1,∞,∞,∞), (0,3,1,6,∞,∞), (0,3,1,1,10,∞), and (0,3,1,1,4,∞). Processing t,z changes nothing. Final π=(NIL,b,s,a,c,NIL). Reverse t,c,a,b,s to obtain s,b,a,c,t, costing 4.

A08-S06

pass	d(s,a,b,c,t,z)	π(s,a,b,c,t,z)
0	(0, ∞, ∞, ∞, ∞, ∞)	(NIL, NIL, NIL, NIL, NIL, NIL)
1	(0, 4, 1, ∞, ∞, ∞)	(NIL, s, s, NIL, NIL, NIL)
2	(0, 3, 1, 2, 11, ∞)	(NIL, b, s, a, a, NIL)
3	(0, 3, 1, 1, 5, ∞)	(NIL, b, s, a, c, NIL)
4	(0, 3, 1, 1, 4, ∞)	(NIL, b, s, a, c, NIL)
5	(0, 3, 1, 1, 4, ∞)	(NIL, b, s, a, c, NIL)

Topological edge order resolves all incoming dependencies before outgoing arcs, so one pass can propagate across an entire shortest path. Our fixed reverse-oriented order reaches t=4 only in pass 4; pass 5 makes no changes.

A08-S07

After pass 5, d=(0,−9,−6,−4,∞,∞) and π=(NIL,b,a,b,NIL,NIL). Only a→b is relaxable in a read-only check, with candidate −8<−6. Seed b, search original arcs onward to obtain {b,a,c}, and assign −∞ to all three. Marking only the cycle incorrectly leaves c finite.

A08-S08

Induct on shortest-path edge count. Its prefix of k−1 arcs is correct by the previous pass, and pass k scans the last arc. For the false equality claim, scan s→a:1 then a→b:1. Pass 1 yields d[b]=2 even though no one-arc s-to-b walk exists. A decrease-key binary heap uses O((V+E)log V) time and O(V) heap space. A lazy heap uses O((V+E)log(V+E)) time and O(V+E) heap space; skip entries whose key differs from current d or whose vertex is already settled.

A08-P01 States and bases

L limits edge count; D limits the allowed intermediate set. Zero edges permit only the empty path, giving I. Zero intermediates still permit a direct edge, giving W. Set the diagonal to min(0,w(i,i)); do not erase a negative self-loop.

A08-P02 Min-plus and stopping

Split after min(a,the walk length) edges; converse concatenation proves the budget equation. Both associative expressions equal $\min_{p,\varphi}(A[i,p]+B[p,q]+C[q,j])$. The requested values are 7 and 3. A directed n-cycle with every edge −1 has no negative closed walk of at most n−1 edges. Its diagonal remains 0 at that budget; use at least n.

A08-P03 Floyd stages k=2 and k=3

At k=2, only (1,3):8→1 and (4,3):10→−1 improve, both with predecessor 2. At k=3, only (1,4):7→3 and (2,4):5→0 improve, both with predecessor 3.

$D^{(2)}$

i \ j	1	2	3	4
1	0	3	1	7
2	∞	0	-2	5
3	∞	∞	0	2
4	2	1	-1	0

$D^{(3)}$

i \ j	1	2	3	4
1	0	3	1	3
2	∞	0	-2	0
3	∞	∞	0	2
4	2	1	-1	0

Path 1→2→3→4 needs intermediate 3, which is outside S_2.

A08-P04 Loop order and predecessors

At i=1,j=2, D[1,3]=∞ and D[4,2]=∞, so no k improves the cell. Only at j=3 does k=4 yield D[1,3]=2. The pass never revisits (1,2). In G, backward predecessors for 3→1 are 1←4←3, giving 3→4→1 of weight 4. For 1→4 they are 4←3←2←1, giving weight 3. The new suffix is a complete k-to-j shortest path. The immediate predecessor of j is Π[k,j], which may differ from k: a suffix k→a→j must store a.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A08-P05 Negative-cycle influence

Cycle 2→3→2 has weight −1. The affected set is {(1,2),(1,3),(1,4),(2,2),(2,3),(2,4),(3,2),(3,3),(3,4)}. $\delta(1,1)=0$, $\delta(4,4)=0$, and $\delta(4,1)=\infty$. A pair is affected exactly when some t is reachable from i, has $D[t,t]<0$, and can reach j. Repeating the negative cycle lowers cost without bound. Do not mark the entire matrix $-\infty$.

A08-P06 Johnson potentials and nonnegativity

$h=(0,0,-2,0)$. In edge order (12,13,14,23,24,34,41,42), $\hat{w}=(3,10,7,0,5,0,2,1)$. Rearranging $h(v)\leq h(u)+w(u,v)$ yields $\hat{w}\geq 0$. The super-source reaches every vertex and therefore every negative cycle. Source 1 can inspect only its reachable region.

A08-P07 Order, restoration, and a false shortcut

$\Sigma(h(v_{a-1})-h(v_a))$ leaves $h(\text{start})-h(\text{end})$, a constant for fixed endpoints. $\delta(1,3)=3-0+(-2)=1$; $\delta(3,1)=2-(-2)+0=4$. Counterexample to adding a constant: s→t=3, s→a=1, a→t=1. The two-edge path is initially shorter at 2. With C=2, direct weight is 5 while the two-edge weight is 6. A cycle has equal endpoints, hence zero net potential shift and unchanged weight.

A08-P08 Choose by density

For sparse $m=\Theta(n)$, Floyd takes $\Theta(n^3)$; both Johnson heap choices have $O(n^2 \log n)$ bounds. For dense $m=\Theta(n^2)$, Floyd takes $\Theta(n^3)$, binary-heap Johnson has upper bound $O(n^3 \log n)$, and Fibonacci-heap Johnson has $O(n^3)$. Full output is $\Theta(n^2)$ for all. Floyd needs $\Theta(n^2)$ working space for current distances and predecessors, or $\Theta(n^3)$ if all n+1 stages are stored. If only selected sources are needed, a full APSP output cost is not the appropriate comparison.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapters 23–25. MIT 6.046J, Fall 2004, Lectures 13–16; Fall 2005, Lectures 16–19. Original handwritten algorithms notes (演算法_wjungle). These notes provide selected proof ideas and context; complete worked traces and diagrams are original educational synthesis.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study